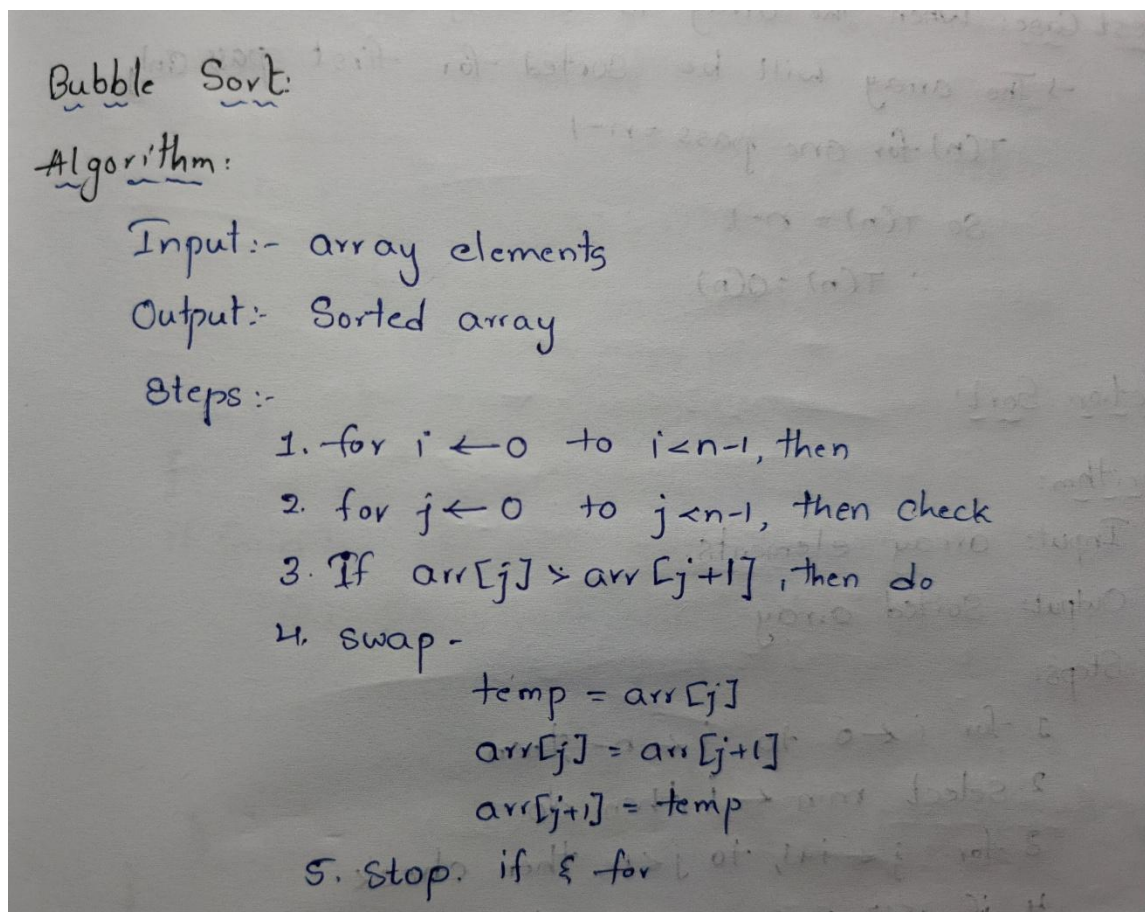


PRACTICAL - 1

AIM : Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

BUBBLE SORT ALGORITHM



PROGRAM:

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {

            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    cout << "Sorted array: ";

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

OUTPUT :

```
PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if ($?) { g++ bubblesort.cpp -o bubblesort } ; if ($?) { .\bubblesort }
Enter number of elements: 5
Enter 5 elements: 23 11 45 98 57
Sorted array: 11 23 45 57 98
PS C:\Users\prash\Downloads\DAA Lab>
```

TIME COMPLEXITY :

Worst and Average cases:

Time Complexity: (Avg & Worst Case)

```
for (int i = 0; i < n-1; i++) { → n-1
    for (int j = 0; j < n-1; j++) { → n-1
        if (arr[j] > arr[j+1]) { → 1
            temp = arr[j]; → 1
            arr[j] = arr[j+1]; → 1
            arr[j+1] = temp; → 1
        }
    }
}
```

$$\Rightarrow T(n) = (n-1)(n-1) \times 1 \times 1 \times 1 \times 1$$

$$= n^2 - 2n + 1 \Rightarrow n^2$$

$$\therefore T(n) = O(n^2)$$

Best case:

Best Case: When the array is already sorted.
→ The array will be sorted for first pass only
 $T(n)$ for one pass = $n-1$
So $T(n) = n-1$
 $\therefore T(n) = O(n)$.

INSERTION ALGORITHM :

Insertion Sort:

Algorithm:

Input: Array Elements.

Output: Sorted Array.

Steps:

1. for $i \leftarrow 1$ to $i \leq n$, then
2. $\text{temp} \leftarrow a[i]$ and $j \leftarrow i-1$ then
3. While $j \geq 0$ and $a[j] > \text{temp}$.
4. $a[j+1] \leftarrow a[j]$ and $j--$ (i.e. $j = j-1$)
5. Stop while.
6. $a[j+1] \leftarrow \text{temp}$ and
7. Stop for.

PROGRAM:

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

OUTPUT :

```
PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if ($?) { g++ insertionsort.cpp -o insertionsort } ; if ($?) { .\insertionsort }  
Enter number of elements: 5  
Enter 5 elements: 45 67 93 23 89  
Sorted array: 23 45 67 89 93  
PS C:\Users\prash\Downloads\DAA Lab>
```

TIME COMPLEXITY :

average case and worst case :

Time Complexity:- (Worst & Avg Case)

```
for (i=1; i<n; i++) {  
    temp = a[i];  
    j = i-1;  
    while (j >= 0 && arr[j] > temp) {  
        a[j+1] = a[j];  
        j--;  
    }  
    a[j+1] = temp;  
}
```

$\Rightarrow T(n) = n \times n \times 1 \times 1 \times 1 \times 1$
 $T(n) = O(n^2)$

Best case:

Best Case:

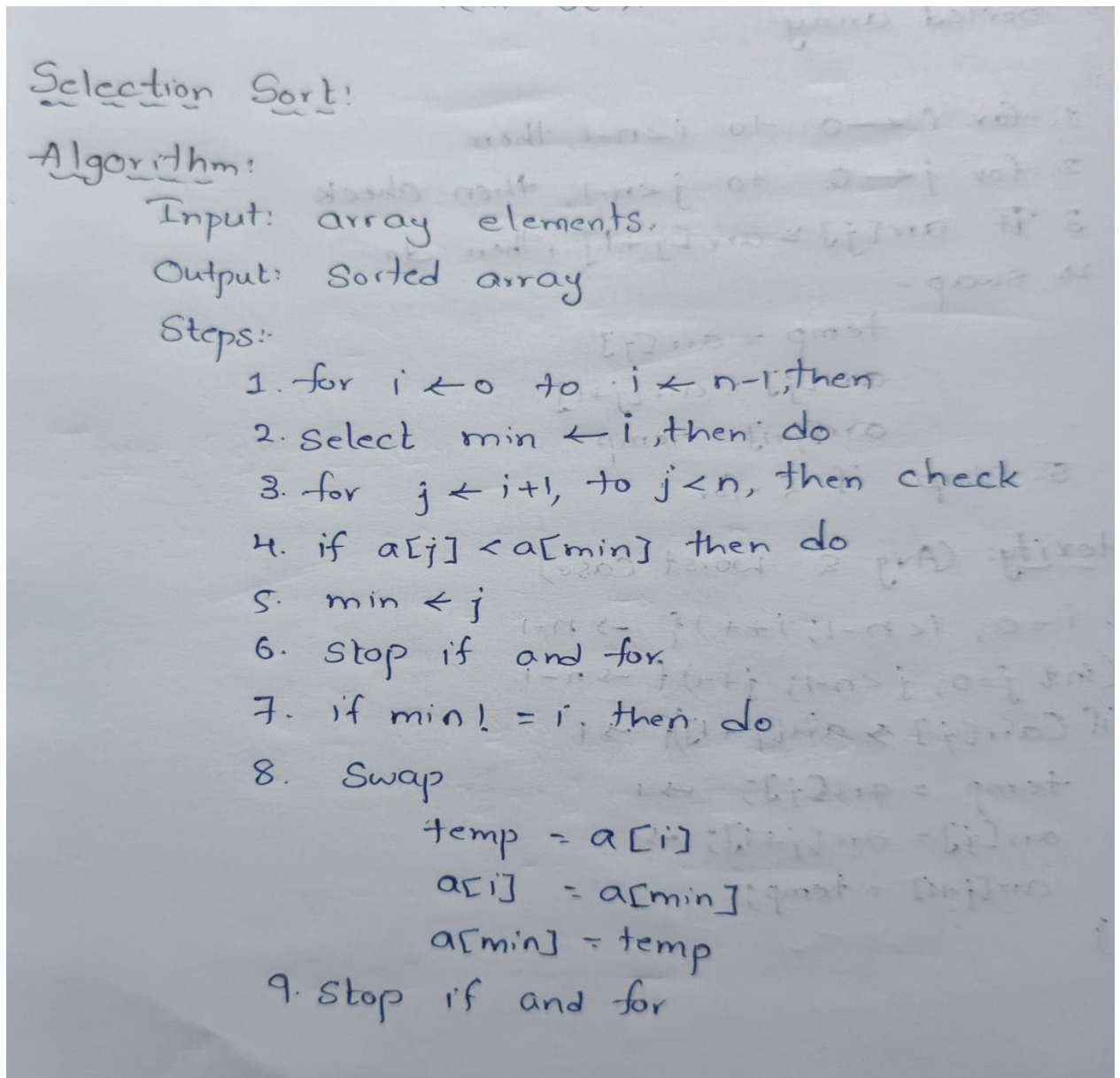
No: of Comparisions for

2^{nd} element = 1
 3^{rd} element = 1
 $\vdots$
 n^{th} element = 1

} Because the array will be already sorted

$\therefore T(n) = n-1$
 $T(n) = O(n)$

SELECTION SORT ALGORITHM :



PROGRAM:

```
#include <iostream>
using namespace std;

class SelectionSort { public:
int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array
size: ";
        cin >> size;

        cout << "Enter array elements: ";

        for (i = 0; i < size; i++)
        {
            cin
>> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }

        cout << endl;
    }

    void selectionsortlogic()
    {
        for (i = 0; i < size - 1; i++)
        {
            int
min = i;

            for (j = i + 1; j < size; j++)
```

```
        {           if (arr[j] <
arr[min])           {
min = j;
        }
    }
    if (min != i)
    {
        temp = arr[i];
arr[i] = arr[min];
arr[min] = temp;
    }
}
};
```

```
int main()
{
    SelectionSort ss;

    ss.arrayinput();

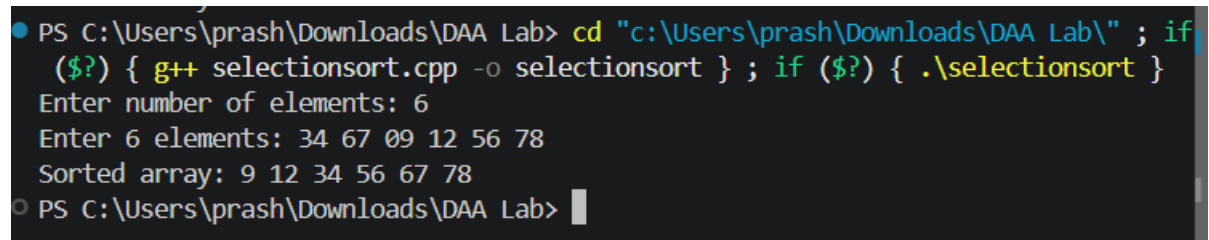
    cout << "Before sorting: ";
    ss.printarray();

    ss.selectionsortlogic();

    cout << "After sorting: ";
    ss.printarray();

    return 0; }
```

OUTPUT :



```
PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if
($?) { g++ selectionsort.cpp -o selectionsort } ; if ($?) { .\selectionsort }
Enter number of elements: 6
Enter 6 elements: 34 67 09 12 56 78
Sorted array: 9 12 34 56 67 78
PS C:\Users\prash\Downloads\DAA Lab>
```

TIME COMPLEXITY :

best case, worst case, average case:

Time Complexity: (Best, Avg & Worst Case)

```

for (i = 0; i < n-1; i++) { → n-1
    int min = i;
    for (j = i+1; j < n; j++) {
        if (arr[j] < arr[min]) { → 1
            min = j; → 1
        }
    }
    if (min != i) { → 1
        int temp = arr[i]; → 1
        arr[i] = arr[min]; → 1
        arr[min] = temp; → 1
    }
}

```

$T(n) = (n-1) (n) \times 1 \times 1 \times 1 \times 1 \times 1 \times 1$
 $= n^2 - n \times 1$
 $T(n) = O(n^2)$

In Selection Sort:-
 Best = Average = Worst = $O(n^2)$.

Merge sort Algorithm:

Merge Sort:

Algorithm:

Input: Array Elements

Output: Sorted Array

Steps:

1. Merge (arr, lb, mid, ub) then

2. Set $i = lb$, $j = mid + 1$, $k = lb$

3. While $i \leq mid$ and $j \leq ub$, then

4. If $arr[i] \leq arr[j]$, then

$temp[k] = arr[i]$

$i = i + 1$

else

$temp[k] = arr[j]$

end if

$k = k + 1$

End while.

5. While $j \leq ub$, then

$temp[k] = arr[j]$

$j = j + 1$

$k = k + 1$

end while

6. While $i \leq mid$ then

$temp[k] = arr[i]$

$i = i + 1$

$k = k + 1$

end while.

7. for $i \leftarrow lb$, $k \leftarrow 0$ to $i \leq ub$, then

$arr[i] = temp[k]$

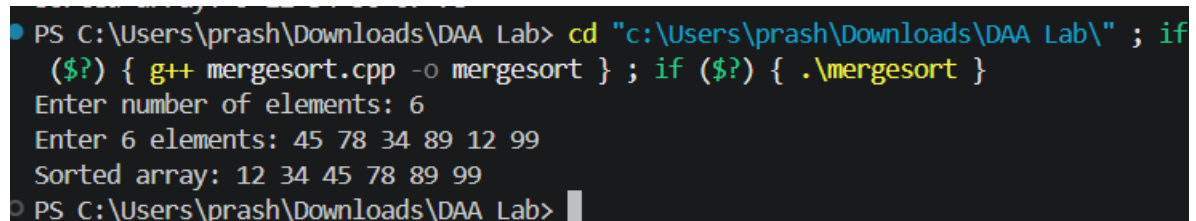
Stop for.

PROGRAM:

```
#include <iostream> using
namespace std;
void merge(int arr[], int low, int mid, int high) {
    int i = low;    int j = mid
+ 1;    int k = 0;    int
temp[high - low + 1];
    while (i <= mid && j <= high) {
        if (arr[i] < arr[j]) {
temp[k] = arr[i];
            i++;        }
        else {            temp[k] =
arr[j];
            j++;
        }        k++;
    }
    while (i <= mid) {
temp[k] = arr[i];
        i++;        k++;
    }
    while (j <= high) {
temp[k] = arr[j];
        j++;        k++;
    }
    for (int i = low, k = 0; i <= high; i++, k++) {
        arr[i] = temp[k];
    }
}
void mergeSort(int arr[], int low, int high) {
    if (low < high) {        int mid =
(low + high) / 2;        mergeSort(arr,
low, mid);        mergeSort(arr, mid
+ 1, high);
```

```
        merge(arr, low, mid, high);
    }
} int main()
{   int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {        cin >>
arr[i];
    }   mergeSort(arr, 0, n -
1);   cout << "Sorted
array: ";   for (int i = 0; i <
n; i++) {        cout << arr[i]
<< " ";
    }
return 0;
}
```

OUTPUT :



```
PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if
($?) { g++ mergesort.cpp -o mergesort } ; if ($?) { .\mergesort }
Enter number of elements: 6
Enter 6 elements: 45 78 34 89 12 99
Sorted array: 12 34 45 78 89 99
PS C:\Users\prash\Downloads\DAA Lab>
```

TIME COMPLEXITY :

best case & average case & worst case :

Time Complexity: (Best, Average & Worst):

merge sort (arr, lb, mid); $\rightarrow n/2$
 merge sort (arr, mid+1, ub); $\rightarrow n/2$
 mergesort (arr, lb, mid, ub); $\rightarrow n$

$$T(n) = T(n/2) + T(n/2) + n$$

$$T(n) = 2T(n/2) + n$$

Master Method:

$$T(n) = aT(n/b) + f(n)$$

$a = 2, b = 2, f(n) = n, p = 0, k = 1$

$$n^{\log_a b} = n^{\log_2 2} = n$$

$$f(n) = n$$

$$n^{\log_2 2} = n \quad \therefore \log_a b = k$$

Case : 2: $p > -1$

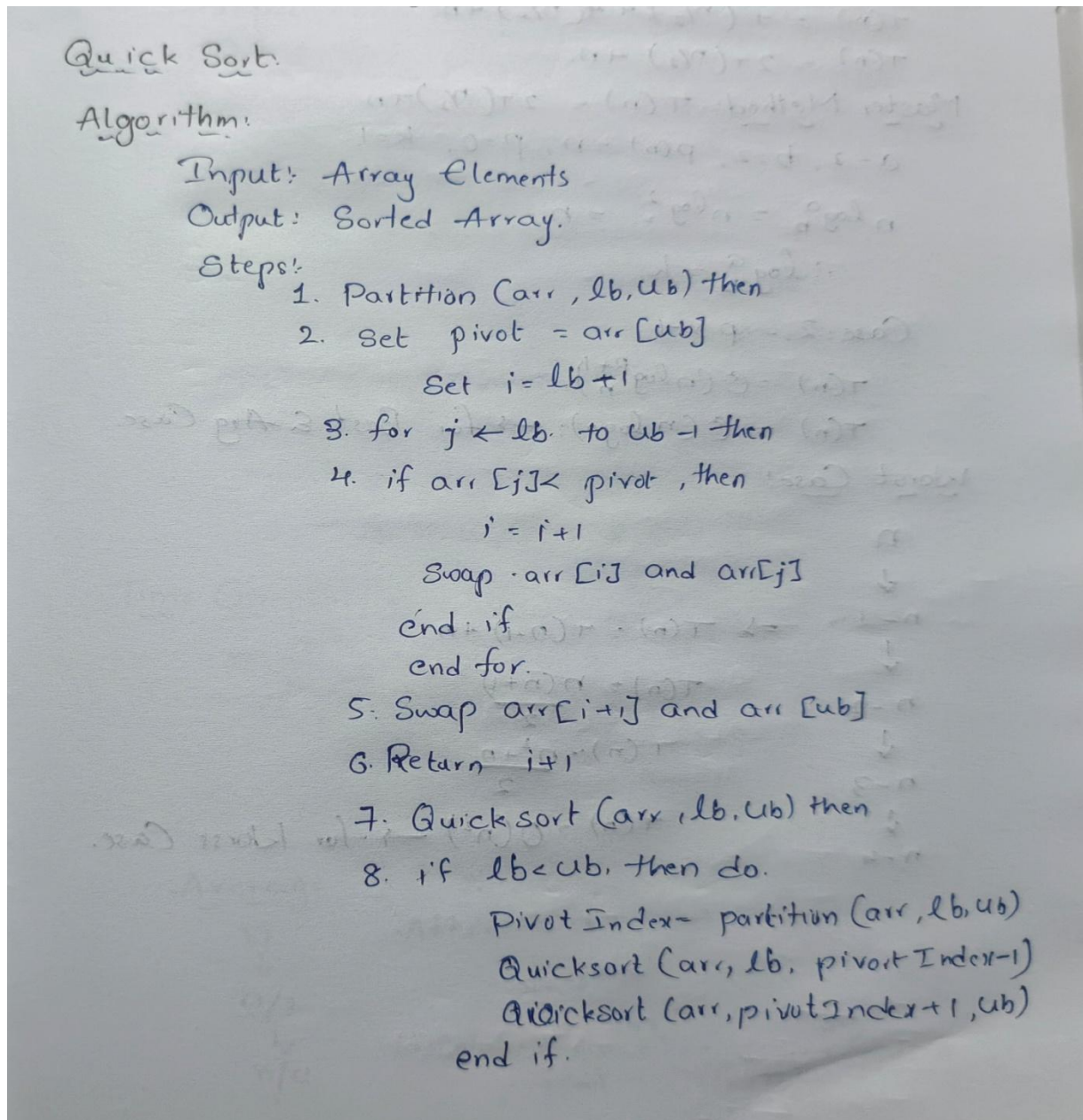
$$T(n) = O(n \log^{p+1} n)$$

$$T(n) = O(n \log n)$$

In merge sort:

$$\text{Best} = \text{Average} = \text{Worst} = O(n \log n)$$

QUICK SORT ALGORITHM:



PROGRAM :

```
#include <iostream> using namespace
std; int partition(int arr[], int low, int
high) {    int pivot = arr[high];    int i =
low - 1;
    for (int j = low; j < high; j++) {

        if (arr[j] < pivot) {
i++;        int temp =
arr[i];        arr[i] =
arr[j];        arr[j] =
temp;
        }
    }    int temp = arr[i
+ 1];    arr[i + 1] =
arr[high];    arr[high] =
temp;    return i + 1; }
void quickSort(int arr[], int low, int high) {
if (low < high) {    int pi = partition(arr,
low, high);    quickSort(arr, low, pi - 1);
    quickSort(arr, pi + 1, high);
}
} int main()
{    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";    for
(int i = 0; i < n; i++) {        cin >> arr[i];
    }    quickSort(arr, 0, n -
1);    cout << "Sorted
array: ";    for (int i = 0; i <
n; i++) {        cout << arr[i]
<< " ";
```

```
}  
return 0;  
}
```

OUTPUT :

```
● PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if  
  ($?) { g++ quicksort.cpp -o quicksort } ; if ($?) { .\quicksort }  
  Enter number of elements: 8  
  Enter 8 elements: 14 35 78 23 67 99 42 55  
  Sorted array: 14 23 35 42 55 67 78 99  
○ PS C:\Users\prash\Downloads\DAA Lab> █
```

TIME COMPLEXITY : best case & average case & worst case:

Time Complexity : (Best & Average Case)

Quicksort (arr, lb, pivot Index -1) $\rightarrow n/2$
 Quicksort (arr, pivot Index +1, ub) $\rightarrow n/2$
 Partition (arr, lb, ub) $\rightarrow n$
 $T(n) = T(n/2) + T(n/2) + n$
 $T(n) = 2T(n/2) + n$

Master Method: $T(n) = 2T(n/2) + n$
 $a = 2, b = 2, p(n) = n, p = 0, k = 1$
 $n \log_a b = n \log_2 2 = n$
 $\therefore \log_a b = k$

Case: 2:- $p > -1$
 $T(n) = O(n \log^{p+1} n)$
 $T(n) = O(n \log n)$ $\rightarrow$ for Best & Avg Case

Worst Case:

n
 $\downarrow$
 $n-1$ $\Rightarrow T(n) = T(n-1) + n$
 $\downarrow$
 $n-2$ $T(n) = \frac{n(n+1)}{2}$
 $\downarrow$
 $n-3$ $T(n) = \frac{n^2 + n}{2}$
 $\downarrow$
 $n-4$ $T(n) = O(n^2) \rightarrow$ for Worst Case.

Conclusion :

The sorting algorithms Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort were successfully implemented and analyzed. Each algorithm has a different way of sorting elements and different time requirements. Merge Sort and Quick Sort are generally faster for large data, while Bubble, Selection, and Insertion Sort are simple and easy to understand.